LEXIGRID

Record Editing and Review

Version 5.54.15 | Reviewed 2026-11-11

Shahryar Jahangir, Aniza Inc.

Procedures, examples, verification and operational limits. Complete commissioning on your deployment before serving customer traffic. Sample values are fictional; original keys and data must be preserved.

Contents

Editing records in LexiGrid 5.54.15 **3**

- Risk example . . . 3
- Control example . . . 3
- Framework example . . . 3
- Policy example . . . 3
- Enterprise IRM example . . . 3
- Security and operational notes . . . 4

Editing records in LexiGrid 5.54.15

The Risks, Controls, Frameworks, Policies, and Enterprise IRM workspaces now provide Edit actions. Framework requirements can be added and edited in Frameworks. These changes do not enable arbitrary editing of approvals, signed evidence, or immutable audit history.

Risk example

1. Sign in as a tenant administrator, risk manager, or risk editor.

2. Open Risks, locate the risk, and select Manage, then Edit risk.

3. Change its title, description, category, owner, likelihood, impact, or non-closed status.

4. Select Save changes. LexiGrid recalculates risk scoring in its existing API and records the update in its audit history.

5. If another user has updated the record, refresh the page and retry. LexiGrid rejects stale versions rather than overwriting newer changes.

6. To close a risk, use Resolve & Close and supply a justification. Editing cannot bypass the closure workflow.

Example: Change an authentication risk likelihood from 4 to 3 after documenting effective compensating controls. Reassess with AI separately; AI recommendations do not automatically change the official rating.

Control example

1. Open Controls and select Edit on the relevant control.

2. Change the code, title, description, owner, or lifecycle status.

3. Save. Duplicate codes are rejected. Stale versions are rejected with a conflict error.

4. Verify the revised control in the list and the corresponding operation in Audit.

Framework example

1. Open Frameworks and select Edit.

2. Change the framework name or revision, then Save changes.

3. Duplicate name/revision combinations are rejected. Existing linked requirements remain attached to the same framework ID.

4. Select the framework in Requirements, then choose Add requirement or Edit. Save the reference and requirement text.

5. API clients must send the version token returned by GET when PATCHing a framework or requirement. A stale token returns 409; reload and review the newer record before retrying.

Policy example

1. Open Policies and select Edit for a draft policy.

2. Update the code, title, policy text, or owner and Save.

3. Policies under review, approved, or retired cannot be edited directly. This prevents bypassing the approval lifecycle. Return a policy to draft using the governed transition where permitted, or initiate a new controlled revision process.

Enterprise IRM example

1. Open Enterprise IRM and choose the record category.

2. Select Edit beside the desired record; change title, owner, and data fields.

3. Save. The API checks tenant, authorization, record kind, required data fields, and record version.

4. Use Review for lifecycle transitions; the Edit action does not advance workflow state.

Security and operational notes

- Tenant boundaries are enforced server-side.

- Edit permissions are enforced by backend authorization, not only hidden UI buttons.

- Risk, control, policy, and Enterprise IRM edits use optimistic version checks.

- Framework and requirement edits require content-version tokens and reject stale updates atomically.

- Approvals, evidence files, and audit records remain governed by their existing APIs and should not be silently overwritten.

- Vendor, business-service and loss-event data use the Enterprise IRM record editor.

- During upgrades, preserve the database and configuration. The change adds no database migration.

Risk change history

Choose **Manage -> Change history** for a read-only list of committed creation, edits, AI-applied score changes and closure. Expand before/after values and inspect each event's audit signature status. A valid signature does not prove complete history; absent historical values remain absent. API: `GET /api/risks/{id}/history?limit=50&offset=0`.